KB Article Markdown Converter: Design Document

Tenzin Dorjee

Summary

A TypeScript service turns my.opentext.com knowledge articles into clean Markdown with front matter and serves it at `GET /v1/articles/{id}`. Dev testing is complete and designed to scale behind a CDN like Akamai.

The converter reads the article from the site's JSON API, not from the page, because the page is rendered in the browser and its server HTML holds no article text.18 HTML clean-up rules, each written against a real defect in a real article. Every conversion is checked at runtime: the article is always served, and problems are logged with the article ID so the rules can be improved.

Assignment item	What was delivered	Status
Markdown converter	18 HTML cleanup rules, front matter, content preserved, handle attachments,	Built and tested
Tests	Vitest Unit tests per rule, 4 golden files compare quality against,	Run on every push in CI
Expose through an API	Hono app: one route, cache headers, purge tag, quality header and logs	Built, verified live against my.opentext.com
CI workflow (build and test)	GitHub Actions: typecheck, tests, compile, Docker image build	Built, runs on every push and pull request
Serverless + CDN assumption	Multi-stage Docker image for Cloud Run, Akamai caching and purge design	Designed, not yet deployed
Distribution design and evaluation	Distribution diagram, discovery channels, metrics with targets	Designed

The rest of this document explains how the content source was found, the design, the decisions made along the way and the limitation and constraints of the system thus far.

1. Context and problem

AI agents (LLM crawlers, answer engines, coding assistants, and internal RAG/chat tools) are becoming a primary way customers find and use OpenText support content. Our knowledge articles on my.opentext.com, however, are built for browsers: the article body sits inside navigation, scripts, styling, and other UI markup meant for human readers.

Having experience working on WebAI I assumed I could fetch, download and convert the webpage directly. However, the article pages are rendered in the browser. The HTML the server returns for `/support/article/{id}` return only a shell, until JavaScript fetches the article from a JSON endpoint and inserts HTML into the browser. That causes 3 main problems for AI clients:

- **Invisible content.** A crawler that does not run JavaScript receives the empty shell. I saw that the sitemap lists product pages, not articles, so there is also no simple way to discover them.
- **Cost and accuracy.** A client that does render the page spends most of its token budget on markup, which raises cost and dilutes what it extracts.
- **Staleness.** Expensive pages are re-fetched less often, so AI tools may answer from outdated copies.

Markdown fits this better than HTML. It keeps the structure that carries meaning and drops scripts, styling and layout. The industry is moving the same way: in February 2026, Cloudflare launched "Markdown for Agents," which converts HTML to Markdown at the edge for AI clients and reported roughly an 80% token reduction on its own blog post. However, these services are expensive and brittle. The aim of this project is addressing that gap with a custom solution, helping OpenText better serve its clients.

2. Goals and non-goals

The goals put faithfulness first: a compact article that silently drops a step is worse than a long one.

Goal	Target	How it is met
G1: compact content	Content is preserved including heading, links, hierarchy, lists, tables, code, and attachments Stripping JS scripts, boilerplate, tags etc…	18 clean-up rules, checks, unified library for html to markdown conversion, golden files
G2 Low latency	CDN hit: about as fast as the HTML page.	Testing manually or tools to test
G3 Freshness	CDN serves an updated article within seconds of publish, clients re-request within 5 minutes	CMS Purge on publish by cache tag, 5-minute client TTL, 24-hour CDN TTL only as a backstop if Purge fails.
G4 Reliability	High availability, and graceful degradation	`stale-if-error` serves the last good copy for a day, errors are never cached, and timeouts on upstream calls
G5 Observability and improvement	Any conversion problem is visible with its article ID on the first request	Checks run on every conversion, one JSON log line per conversion with a quality tier. Article is still delivered.
G6 Public	Never reveal that a private article	Non-public and missing articles return the

Goal	Target	How it is met
content only	exists	same 404

Non-goals

- **NG1 Retrieval by ID only.** No search or listing in the API. Discovery (`llms.txt`, a Markdown sitemap) belongs to distribution, Section 10.
- **NG2 No authentication.** Entitled or internal articles are out of scope.
- **NG3 No rewriting.** Conversion is deterministic and faithful. No LLM summarising, translating or editing.
- **NG4 No changes to the existing site.** The KB authoring system and the human HTML pages stay as they are.
- **NG5 No state in the service.** No database or in-process cache, all caching lives in the CDN, which keeps purging simple.

3. Assumptions and constraints

The biggest assumption is that the front end's article API can be used as a source, the biggest constraint is that this project had no access to OpenText's CMS, CDN or cloud accounts, so those parts are designed rather than built.

Assumptions

#	Assumption	If it is wrong
A1	The article API (`/api/web/support/article/{id}`) returns the same content the page shows. It is an internal endpoint of the front end, not a published API	Production use must be agreed with its owning team, ideally through origin-side access rather than the public edge
A2	An article is public when `CanRead` is `Public_c` and `PhaseId` is `External`	A private article could be served; the rule lives in one place (`convert.ts`), so it is a one-line fix
A3	Article IDs are numeric (1–12 digits), stable, and the same in the page URL and the API	Validation would reject real IDs
A4	Akamai, which already fronts my.opentext.com, is the CDN and the service runs on Google Cloud Run	The Docker image runs unchanged on AWS App Runner or Azure Container Apps. cache headers are standard HTTP
A5	Articles are served in English. The site has 11 locales, localised retrieval was not investigated	A `lang` parameter and front-matter field would be needed
A6	The publishing system can call a hook when an article is published or unpublished	Freshness falls back to the TTLs: up to 24 hours at the CDN instead of seconds (Section 9)

#	Assumption	If it is wrong
A7	Four sample articles are enough to start, the runtime checks find what they do not cover	This held: the first unseen article (1770456) showed three new patterns, all caught by the checks (Section 7)

Constraints

- **C1 Bot protection.** Akamai Bot Manager blocks some non-browser traffic, requests from this project's cloud workspace were refused. CI therefore never calls the live site (tests use saved fixtures), and production should reach the API from inside OpenText's network.
- **C2 No platform access.** No access to the KB CMS, Akamai configuration or a Google Cloud project. Purging, edge routing and deployment are specified, not implemented.
- **Inconsistent HTML.** Articles were written by many authors. The same article numbered its steps three different ways. The converter must tolerate inconsistency rather than assume a template.
- **C4 Unversioned upstream.** The article API can change shape without notice. A zod schema validates every response and turns a mismatch into a clear `502`.

4. Discovery: finding the content source

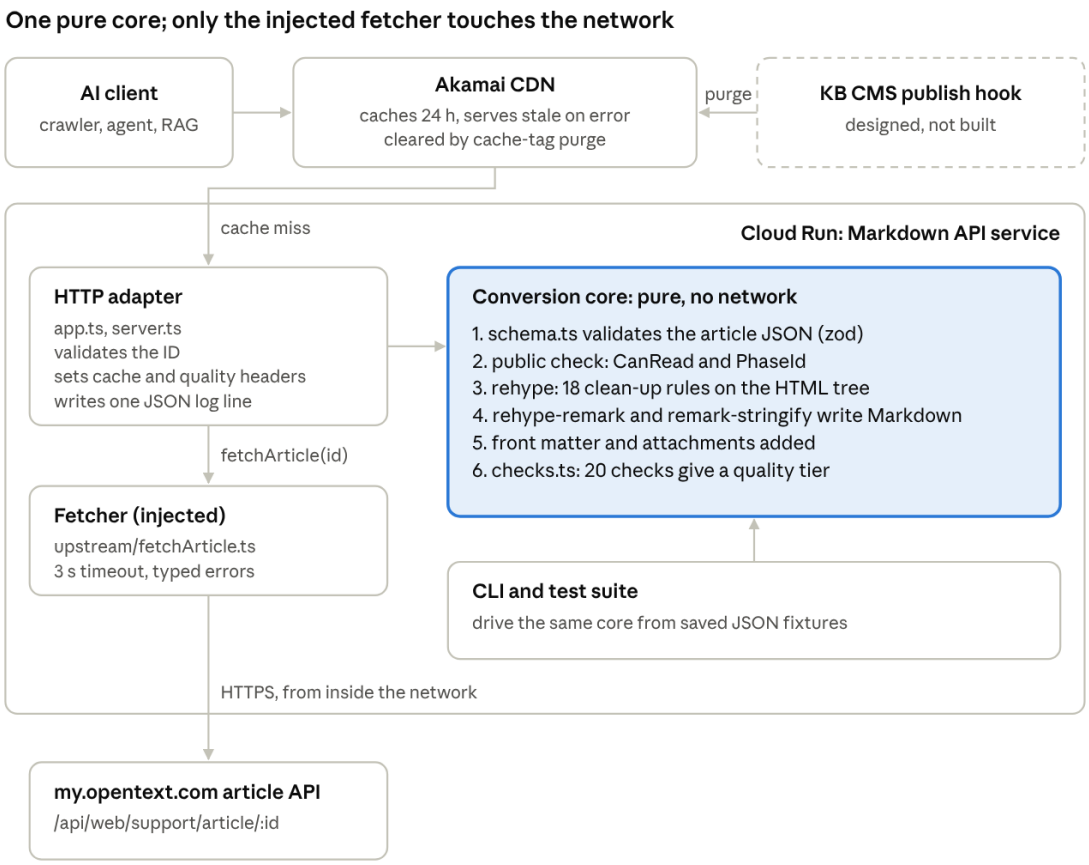
The converter reads the article API because investigation showed the page itself carries no article text. The steps, in the order they were taken:

1. **Fetched the page**. using curl returned a “Loading Article” shell
2. **Watched the network.** DevTools → Network → Fetch/XHR showed the page calling `GET /api/web/support/article/{id}`, which returns a JSON envelope `{ status, success, response }`. `response` holds `Title`, `Content` (the article as an HTML fragment), `Products` with versions, `Version`, `CreatedTime` and `LastContentUpdateTime`, `CanRead`, `PhaseId`, `Attachments`.
3. **Checked it is usable.** The endpoint returns `200` from a plain `curl` the response is not cached at the edge (`Server-Timing: cdn-cache; desc=MISS`).
4. **Ruled out scraping the page.** Rendering it in a headless browser like puppeteer would work, but would add latency and others.
5. **Looked for a list of articles.** The sitemap lists product pages, not articles.
6. **Saved the responses as fixtures.** All four API responses are stored in `tests/fixtures/`. Tests run offline and repeatably, and CI never depends on the live site or on getting past bot protection.

Decision: consume the JSON API directly. It is faster and lighter than rendering, and decouples conversion from the page layout. The risk moves to the API contract changing, which the zod schema and the planned nightly drift check cover.

5. Architecture

The service is a pure conversion core wrapped by thin adapters. The core takes article JSON and returns Markdown with no network, file or clock access. Only the fetcher talks to my.opentext.com, and it is passed into the app rather than imported by it.



Runtime architecture · dashed = designed, not built

The CDN answers most requests. However, on a cache miss, the HTTP adapter asks the injected fetcher for the article and hands it to the core, and the CLI and tests drive the same core from saved files.

Why this shape

- **Offline tests.** The core needs no network, so every rule, golden file and check runs from fixtures, in CI, in seconds.
- **Swappable source.** `server.ts` calls `createApp({ fetchArticle })` with the live fetcher, a test, a staging setup or a fetcher with retries can pass another one without touching `app.ts`.
- **Portable.** Nothing in the conversion path depends on Node-only APIs, so the core could later run at the edge.
- **One gate.** The public-only check lives in the core, so no adapter can bypass it.

Request flow on a cache miss

1. A client requests `/v1/articles/1770456` (in production, the article URL with `Accept: text/markdown`, rewritten by Akamai).
2. The CDN has no fresh copy and forwards the request to Cloud Run.
3. `app.ts` checks the ID is 1–12 digits, otherwise it returns `400` without calling upstream.

4. The fetcher calls the article API with a 3-second timeout and maps failures to typed errors.
5. The core validates, applies the public check, cleans, converts and adds front matter and attachments.
6. The checks produce a quality tier rating and `app.ts` writes one JSON log line.
7. The response goes back with cache headers, the CDN stores it for 24 hours under the tag `article-1770456`.

Code layout

```
src/
├── core/        convert.ts, schema.ts, errors.ts, frontmatter.ts, attachments.ts,
site.ts, checks.ts
│   └── rehype/  index.ts (rule order), utils.ts, rules/ (18 files, one per rule)
├── upstream/    fetchArticle.ts
├── api/         app.ts (routes), server.ts (entry point)
└── cli/         convert.ts, check.ts
tests/
├── fixtures/    4 saved API responses
├── expected/    golden Markdown, one per fixture
└── *.test.ts
Dockerfile
.github/workflows/ci.yml
```

6. Conversion pipeline

The article HTML is parsed into a tree, cleaned by 18 small rules that each fix one real defect, and only then written out as Markdown. A generic HTML-to-Markdown converter alone produced output with escaped steps (`2\.`), glued words ("theID"), prose in code blocks and underlines turned into italics, so the rules are where most of the quality comes from.

Stages

1. **Validate** (`schema.ts`). A zod schema checks the envelope and article and produces a typed: `Article`. `success: false` or a null `response` means not found. `UpstreamFormatError` if upstream convention changed, and 200 success if it gets validated.
2. **Public check** (`convert.ts`). Only `CanRead = Public_c` and `PhaseId = External` pass anything else raises the same `ArticleNotFoundError` as a missing article.
3. **Parse** (`rehype-parse`). `Content` becomes an HTML syntax tree (hast).
4. **Clean** (`rehype/index.ts`). The 18 rules run in a fixed order, below.
5. **Convert** (`rehype-remark`, `remark-gfm`, `remark-stringify`). The tree becomes a Markdown tree and then text: `-` bullets, fenced code blocks, GFM tables.
6. **Front matter** (`frontmatter.ts`). YAML from the article fields: title with whitespace collapsed, ID, canonical URL, type, products with de-duplicated versions, version, created and last-modified as ISO 8601 UTC, source. `Votes` and `Views` are left out: they change on every view and would make every cached copy differ.
7. **Attachments** (`attachments.ts`). An `## Attachments` section with absolute download links and human sizes ("1.8 MB").
8. **Assemble** (`convert.ts`). Front matter, `# Title`, body, attachments. Returns `{ markdown, meta, html }` the source HTML is returned so the checks can compare against it.

The rules, in run order

Order matters, index.ts enforces the order (Each rule file opens with a real before-and-after example from the article).

The unified libraries produced a solid base conversion, but not a perfect one. I edited the Markdown by hand until it was exactly what I wanted and used that as the golden version. I then gave the golden file to an AI assistant and had it write clean-up rules until the converter produced that output on its own, importing those rules into the codebase.

7. Quality assurance

The converter is checked in four layers, and the last one runs in production on every conversion, so an article no test has seen is still checked. The idea: tests prove the rules work on known articles; checks prove the output is sound on any article.

Layer	What it checks	When it runs	Catches
1. Rule unit tests (`rules.test.ts`)	Each rule on its real before-and-after example	Every `npm test`, CI	A rule edit that breaks its own case
2. Golden files (`tests/expected/<id>.md`)	The full output of each fixture. Regression testing	Every `npm test`, CI	Any change to any output, shown as a diff
3. Checks (`checks.ts`), full set	20 post-conditions, including idempotence	Every `npm test` on every fixture, CI	Lost text, broken structure, leftover HTML
4. Checks at runtime	The same checks minus idempotence	Every conversion in the API	Problems in articles no test has seen

The checks. Each one inspects the result independently, by parsing the Markdown or comparing it with the source HTML, rather than re-running the rule it backs and trusting it. They need no expected output, so they apply to any article. Content kept, Structure, Leftovers, Stability, Worth a look

Quality tiers at runtime. The API sums up the checks as one tier, serves the article whatever the tier, and logs it.

Tier	Meaning	Served?	CDN cache	Log severity
`ok`	No issues	Yes	24 h	INFO
`warning`	Worth a look	Yes	24 h	WARNING
`error`	Something is wrong, such as a lost link	Yes	24 h	ERROR
`critical`	More than 20% of the words lost	Yes	1 min, so a fix reaches users quickly	CRITICAL

Serving a flawed conversion is deliberate: an article with one plain-looking list is far more useful to an AI client than an error. Only text loss changes caching, because that is the case where a cached copy is actively misleading.

8. API Contract

The API has one content route, `GET /v1/articles/{id}`, which returns the article as `text/markdown` with headers that tell the CDN how to cache it. A second route, `GET /healthz`, returns `ok` for the platform's health checks.

Success response (live output for 1770456)

```
HTTP/1.1 200 OK
content-type: text/markdown; charset=utf-8
cache-control: public, max-age=300, s-maxage=86400, stale-while-revalidate=86400,
stale-if-error=86400
edge-cache-tag: article-1770456
x-conversion-quality: ok

---
title: CVE-2021-44228, CVE-2021-45046 and CVE-2021-45105 in AutoPass license server
article_id: '1770456'
url: https://my.opentext.com/support/article/1770456
type: Security Bulletins
...
---

# CVE-2021-44228, CVE-2021-45046 and CVE-2021-45105 in AutoPass license server
```

Header	Value	Purpose
`Content-Type`	`text/markdown; charset=utf-8`	What the body is
`Cache-Control`	`public, max-age=300, s-maxage=86400, stale-while-revalidate=86400, stale-if-error=86400`	CDN keeps it 24 h, clients 5 min (Section 9)
`Edge-Cache-Tag`	`article-{id}`	Lets one purge clear every cached variant of the article, Akamai strips it before responding
`X-Conversion-Quality`	`ok`, `warning`, `error` or `critical`	Lets clients and monitoring see conversion quality without parsing logs

Errors. Bodies are JSON, `{ "error": <status>, "message": "…" }`. Each error type is mapped in one place in `app.ts`.

Condition	Status	Cache-Control	Why
ID not 1–12 digits	400	1 min	Rejected before any upstream call, also stops path tricks in the upstream URL

Condition	Status	Cache-Control	Why
Article missing or not public	404	1 min	Same response for both, so private IDs are not revealed, short cache absorbs repeated bad links
Upstream down, 5xx, or unexpected JSON shape	502	`no-store`	Temporary, the CDN serves its last good copy instead
Upstream takes longer than 3 s	504	`no-store`	Same as above
Unexpected bug	500	none set	Not expected, Hono's default handler

At the edge (designed, not built). Akamai would map two public URLs onto the origin route, so AI clients never see `/v1/`: the canonical article URL when the request sends `Accept: text/markdown`, and `/support/article/{id}.md`, which always returns Markdown. Browsers keep getting the existing HTML page. Rewriting to a separate origin path keeps HTML and Markdown in separate cache entries, which avoids relying on `Vary: Accept`, a header CDNs handle inconsistently. Also bot routing rules would route bots

9. Caching and freshness

Freshness comes from purging the CDN when an article is published, the 24-hour CDN lifetime is only the backstop if a purge is missed. That split lets the CDN answer almost every request while still showing an edit within seconds.

The problem this solves. A human opening an article sees the new version as soon as it is published, because their browser calls the article API "directly". A bot reading the Markdown gets whatever the CDN cached. With time-based caching alone, the first draft of this design (15 minutes at the CDN plus an hour of `stale-while-revalidate`) let a bot lag a human by up to 75 minutes. For a security bulletin, that gap matters.

Cache-Control on a successful response

Directive	Value	Who obeys it	Why this value
`public`	—	All caches	Articles are the same for every reader
`max-age`	300 (5 min)	The client's own cache	A purge cannot reach a bot's or browser's local copy, so this is the most a client can lag after an update.
`s-maxage`	86400 (24 h)	The CDN	Cuts upstream fetches to about one per article per day. Assumption is that pages are not frequently updated (<1 daily)
`stale-while-`	86400 (1 day)	The CDN	After expiry, serve the old copy instantly and refresh

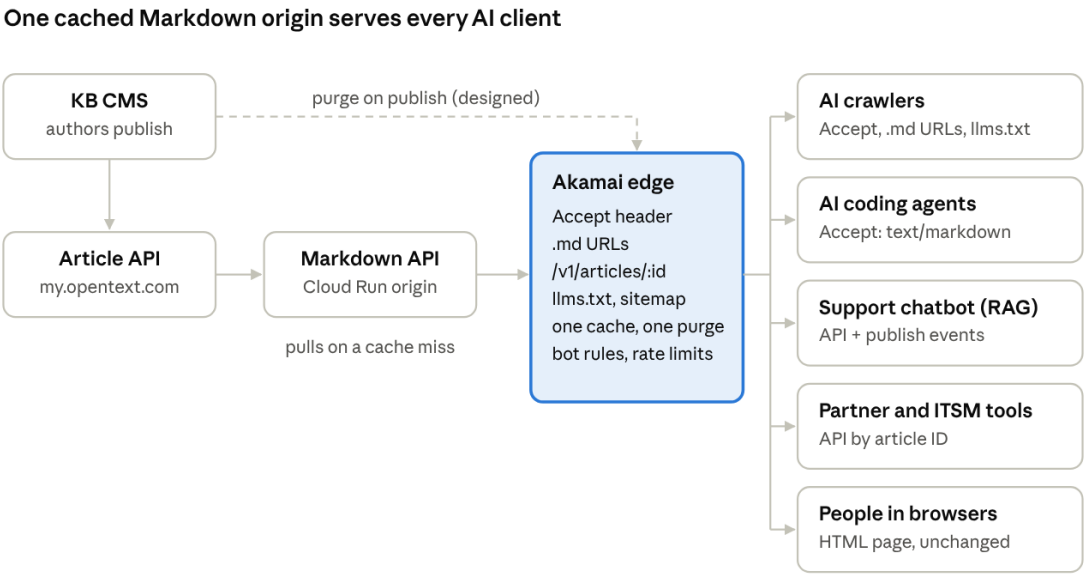
Directive	Value	Who obeys it	Why this value
`revalidate`			in the background, so no reader waits
`stale-if-error`	86400 (1 day)	The CDN	If the service or my.opentext.com is down, keep serving the last good copy instead of an error

Purge on publish (designed, not built)

1. An author publishes or unpublishes an article in the KB CMS.
2. The CMS publish hook calls Akamai Fast Purge for the cache tag `article-{id}`. Every cached variant of the article carries that tag, so one call clears them all. Purging by URL would miss variants nobody listed.
3. If OpenText's own Akamai caches the article API, the hook purges that URL first. Otherwise the converter could fetch the old version and re-cache it for another day.
4. The next request is an ordinary cache miss: the CDN calls the service, which fetches and converts the fresh article. The service never needs to know a purge happened, because the system is pull-based.

10. Distribution to clients and platforms

Each article's Markdown is produced once per version at a single origin and distributed out through Akamai, so every client and platform reads the same output from one cache, cleared by one purge. Clients pull, nothing is copied to each platform separately.



Distribution design: 5 client groups, 1 origin, 1 cache

AI clients reach the same cached Markdown through whichever channel suits them, people keep the existing HTML page, and only the CDN ever calls the origin.

Channels

Channel	How it works	Main users	Status
Content negotiation	The canonical article URL with `Accept: text/markdown`; Akamai rewrites it to the origin path. The convention Cloudflare and several AI agents already use	AI coding agents, crawlers that ask for Markdown	Designed
`.md` URLs	`/support/article/{id}.md` always returns Markdown	Links in docs and tickets, tools that cannot set headers	Designed
Discovery	`<link rel="alternate" type="text/markdown">` in each HTML page; an `llms.txt` at the site root listing key articles by product; a Markdown sitemap	AI crawlers finding articles they were not given	Designed
Direct API	`/v1/articles/{id}`, versioned, cacheable	OpenText's support chatbot and RAG index, partner portals, ITSM	Built
Publish events	On publish, an event (for example a Pub/Sub topic) tells subscribers which article changed, so they re-fetch instead of polling	RAG indexes that must stay current	Future

Controls that apply to every channel

- **Search engines keep indexing the HTML.** Markdown responses should carry `Link: <https://my.opentext.com/support/article/{id}>; rel="canonical"`, so `.md` URLs are not treated as duplicate pages.
- **Bot policy at the edge.** Verified AI crawlers are allowed on Markdown paths, unknown bots are rate-limited, and `robots.txt` states which crawlers may use content for search, answers or training.
- **Same freshness everywhere.** Because every channel reads one cache, one tag purge updates all of them at once.
- **Measured per channel.** Akamai logs record channel and user agent, which feeds the adoption metrics in Section 11.

Why one origin behind a CDN rather than generating files for each platform: one source of truth, one conversion per article version, consistent output for every client, and a single place to fix a rule and purge. New clients need no work on the origin, they only need a channel.

11. Evaluating performance and effectiveness

How to measure effectiveness

Effectiveness is measured the way it was planned for WebAI, the pipeline that converts www.opentext.com pages to Markdown for AI crawlers. There, the Akamai bot observability pipeline (DataStream 2 → Google Cloud Storage → Filebeat → Graylog, with a Verified and AI Crawler Governance dashboard) came first: the same logs that decide which bots get Markdown also prove whether it reached them. Each tool answers a different question.

Tool	Question	What to track
Akamai logs in Graylog (built for WebAI)	Are the intended AI clients actually getting Markdown?	Named bots on Markdown URLs, Markdown vs HTML requests per article, time from publish to a bot's first fetch of the new version, search indexers receiving only HTML
OnCrawl (log-based crawl analytics)	Does Markdown change how bots crawl?	Crawl frequency and depth on Markdown-enabled articles vs a control group, whether every Markdown URL is discoverable, before-and-after trends
Google Search Console	Did it matter, and did it harm anything?	AI Overview and AI Mode impressions per article (the outcome metric, with a lag of weeks), search impressions, clicks and position as a harm check, indexing of the HTML pages
Answer-quality evaluation (new for KB)	Is Markdown better input for an AI answer?	About 50 real support questions answered by the same model from rendered-page text and from Markdown

Test design, carried over from WebAI

- **Treatment and control.** Enable Markdown for a set of articles and pair each with a similar article (same product, type and traffic) left HTML-only. Compare the pairs over several weeks, so changes in AI crawling across the whole site do not get credited to Markdown.
- **Phased rollout.** Launch `.md` URLs, then `Accept: text/markdown`, then the discovery files, so each step has its own before and after.
- **Logs set the priorities.** The same logs show which articles bots request most, which decides which articles to pre-warm after a purge and which to add as fixtures first.

The KB articles live on my.opentext.com, so that hostname must be in the DataStream 2 feed, WebAI covered www.opentext.com and 18 other hostnames. The answer-quality evaluation has no WebAI equivalent: knowledge articles are answers to specific support questions, so whether an AI answers them correctly is the most direct test of the whole project. It also doubles as a regression test for rule changes.

12. CI/CD

Every push to `main` and every pull request runs the typecheck, the full test suite and a compile, then builds the Docker image. Only a green build produces an image. Deployment and a nightly check against the live site are designed as the next two stages.

CI workflow (`.github/workflows/ci.yml`, built)

Job	Steps	Notes
`test`	`checkout` → Node 22 with npm cache → `npm ci` → `npm run typecheck` → `npm test` → `npm run build`	`npm ci` installs the exact locked versions and fails if the lock file is stale
`docker`	`checkout` → `docker build -t kb-markdown:<commit sha> .`	`needs: test`: skipped when tests fail, the tag ties each image to its commit

CI runs fully offline: tests use the saved fixtures, so a run never depends on my.opentext.com being up or on GitHub's servers getting past bot protection.

The workflow was tested both ways. A normal push went green. A branch with a deliberately broken golden file turned the `test` job red and skipped `docker`.

Docker image (`Dockerfile`, built)

- **Two stages.** The build stage installs everything and compiles TypeScript to `dist/`. The runtime stage installs production dependencies only and copies `dist/`. No TypeScript, tests or dev tools ship, so the image is smaller, starts faster on Cloud Run and has less to patch.
- **Plain Node at runtime.** `node dist/api/server.js`, not `tsx`, so nothing compiles on start.
- **Non-root.** `USER node`.
- **Port.** `PORT=8080`, which Cloud Run sends traffic to, the server reads it.

Next stages (designed, not built)

- **Deploy (`deploy.yml`, on merge to `main`).** Build and push the image to Artifact Registry, then `gcloud run deploy` with `--min-instances 0` (or 1 in production to avoid cold starts), `--max-instances 10` (caps cost and load on my.opentext.com) and `--memory 512Mi`. Authenticate with Workload Identity Federation, so no long-lived keys sit in GitHub secrets.
- **Nightly drift check.** From inside OpenText's network, fetch a sample of live articles, run the full checks and the schema, and alert when warnings, unknown tags or schema failures appear. New problem articles are saved as fixtures. This finds HTML changes before users do, and is how the next 1770456 would surface without anyone requesting it.

13. Limitations and known issues

The largest limitation is coverage: the rules are proven on four articles, and the knowledge base holds thousands written by many authors. The runtime checks are what make that acceptable, because they report each new pattern instead of hiding it.

Conversion

- **Four fixtures.** Three Knowledge articles and one Security Bulletin. Other article types, older articles and other authors' habits are untested.
- **Some hand-numbered steps stay as lines.** Steps written as bold numbers joined by line breaks inside one paragraph ("**2.** Navigate…", 1770456) and "1) … 2) …" lines inside one paragraph are left as lines. They read correctly but are not a Markdown list.
- **Screenshots carry information the text does not.** Images become links with the alt text "image", an AI client reading the Markdown does not see what the screenshot shows.
- **Tables are flattened.** GFM tables cannot hold lists or merged cells, so complex tables lose some layout. A line break inside a cell becomes a space, which in 1770456 splits a CVSS vector into `AV:N/AC:L/PR:N/UI:N/S: C/C:H/I:H/A:H`. The author wrapped the string mid-token, a rule that joins breaks inside identifier-like tokens would fix it.
- **Front matter passes values through.** Product versions come as the API gives them, including entries such as `Any`, `All`, `1` and `10.70 and`.
- **Unusual tags.** `<video>` becomes an empty link and `<style>` text is dropped. Neither appears in the sampled articles, and the checks flag both.

Service

- **Upstream dependency.** The article API is internal to the site and behind bot protection. There is no agreement with its owners yet, and no retry on transient failures.
- **Runtime checks cost more than the conversion.** 18–60 ms per conversion against 6–26 ms. Acceptable at roughly one conversion per article per day, worth revisiting if purges become frequent.
- **No API route tests and no linter.** Routes were verified by hand against the live site, the core has unit, golden and check tests.
- **English only.** The 11 site locales were not investigated.

Platform

- **Not deployed.** Cloud Run deployment, Akamai rewrites, cache tags and Fast Purge are specified but untested end to end.
- **Freshness depends on a publish hook** that does not exist yet. Without it, the 24-hour TTL can leave bots up to about two days behind (Section 9).
- **No list of all articles.** The search API mixes content types, so a Markdown sitemap, `llms.txt` and bulk pre-rendering need a knowledge-only listing first.

Process

- **Golden files trust the reviewer.** `npx vitest run -u` accepts whatever the converter now produces, a broken rule accepted without reading the diff becomes the new expected output.
- **Line endings.** Golden files compare bytes, so a Windows checkout with CRLF endings can fail in CI; a `.gitattributes` with `* text=auto eol=lf` prevents it.

14. Future work

The next steps are ordered by what most improves what AI clients receive, starting with what is needed to run this in production.

1. **Agree access to the article API** with its owning team, ideally an origin-side route that bypasses bot protection, plus a stable contract or change notice.
2. **Publish hook to Akamai Fast Purge** by tag, purging the upstream API URL first. Required before relying on the 24-hour CDN TTL.
3. **Deploy:** `deploy.yml` to Cloud Run with Workload Identity Federation, Akamai rewrites for `Accept: text/markdown` and `.md` URLs, branch protection on `main`.
4. **Nightly drift check** over a rotating sample of live articles, failures become fixtures.
5. **Discovery files:** an `llms.txt` and a Markdown sitemap, once a knowledge-only article listing exists: `<link rel="alternate" type="text/markdown">` on each HTML page.
6. **Locales:** a `lang` parameter and front-matter field for the site's other 10 languages.